# NYCU DLP LAB1-Backpropagation

學生:楊正豪 學號:314553044

## 1.Introduction

In this lab ,I only use Python standard libraries and Numpy implements a neural network.Based on the results,I will discuss the model's performance on XOR and linear datasets using different activation function and optimizers.

## 2.Experiment Setups

### A.Sigmoid function

```
    def sigmoid(self,x):
        return 1/(1+np.exp(-x))
    def sigmoid_derivative(self, x):
        return x*(1-x)
```

- sigmoid function 將輸入壓縮到0,1之間，讓model學習非線性資料

### B.Neural Network

- 實作input layer、2 hidden layer、output layer，在**init()**中，先設定input size、hidden layer size、output layer size....。接著初始化weight，並將learning rate、activation function、optimizer儲存起來，方便後續更新使用

```
def __init__(self, input_size=2, hidden1_size=5, hidden2_size=5, out_size=1,
             learning_rate=0.001, activation="sigmoid", optimizer="no"):
    self.input_size = input_size
    self.hidden1_size = hidden1_size
    self.hidden2_size = hidden2_size
    self.output_size = out_size
```

```
    # weights
    self.weight1 = np.random.randn(self.input_size + 1, self.hidden1_size)
    self.weight2 = np.random.randn(self.hidden1_size + 1, self.hidden2_size)
    self.weight3 = np.random.randn(self.hidden2_size + 1, self.output_size)
    self.learning_rate = learning_rate
    self.activation = activation
```

```
    self.optimizer = optimizer
```

- 在 forward() 中，資料會由 input layer 開始，依序傳遞到 hidden layer 1、hidden layer 2，最後輸出到 output layer。為了加入 bias，會在每層輸出補上常數 1，之後再與對應的 weight matrix 做矩陣乘法。

```
 def forward(self,x):
        #y=wx+b
    if self.activation=="sigmoid":
            act=self.sigmoid
    elif self.activation=="none":
            act=self.none
    elif self.activation=="relu":
            act=self.relu

    self.x_bias=np.append(x,np.ones((x.shape[0],1)),axis=1)
    self.hidden1_output=act(np.dot(self.x_bias,self.weight1))


self.hidden1_bias=np.append(self.hidden1_output,np.ones((self.hidden1_output.s
hape[0],1)),axis=1)
    self.hidden2_output=act(np.dot(self.hidden1_bias,self.weight2))


self.hidden2_bias=np.append(self.hidden2_output,np.ones((self.hidden2_output.s
hape[0],1)),axis=1)
    self.y=act(np.dot(self.hidden2_bias,self.weight3))
    return self.y
```

## C.Backpropagation

- 在Backpropagation 先決定使用哪個activation，再去計算誤差，在透過MSE計算方式算出y的梯度，而在算error時，在算每個神經元對loss貢獻多大，所以bias要先去掉，接著透過chain rule向前計算各層的梯度，更新weight和bias
- chain rule:要知道某一層weight對loss影響多大，就得全部拆開算，而以下就是對Loss function展開的例子

$$
\begin{aligned}
\frac{\partial L(\theta)}{\partial w_1} &= \frac{\partial y}{\partial w_1}\frac{\partial L(\theta)}{\partial y} \\
&= \frac{\partial x''}{\partial w_1}\frac{\partial y}{\partial x''}\frac{\partial L(\theta)}{\partial y} \\
&= \frac{\partial z}{\partial w_1}\frac{\partial x''}{\partial z}\frac{\partial y}{\partial x''}\frac{\partial L(\theta)}{\partial y} \\
&= \frac{\partial x'}{\partial w_1}\frac{\partial z}{\partial x'}\frac{\partial x''}{\partial z}\frac{\partial y}{\partial x''}\frac{\partial L(\theta)}{\partial y}
\end{aligned}
$$

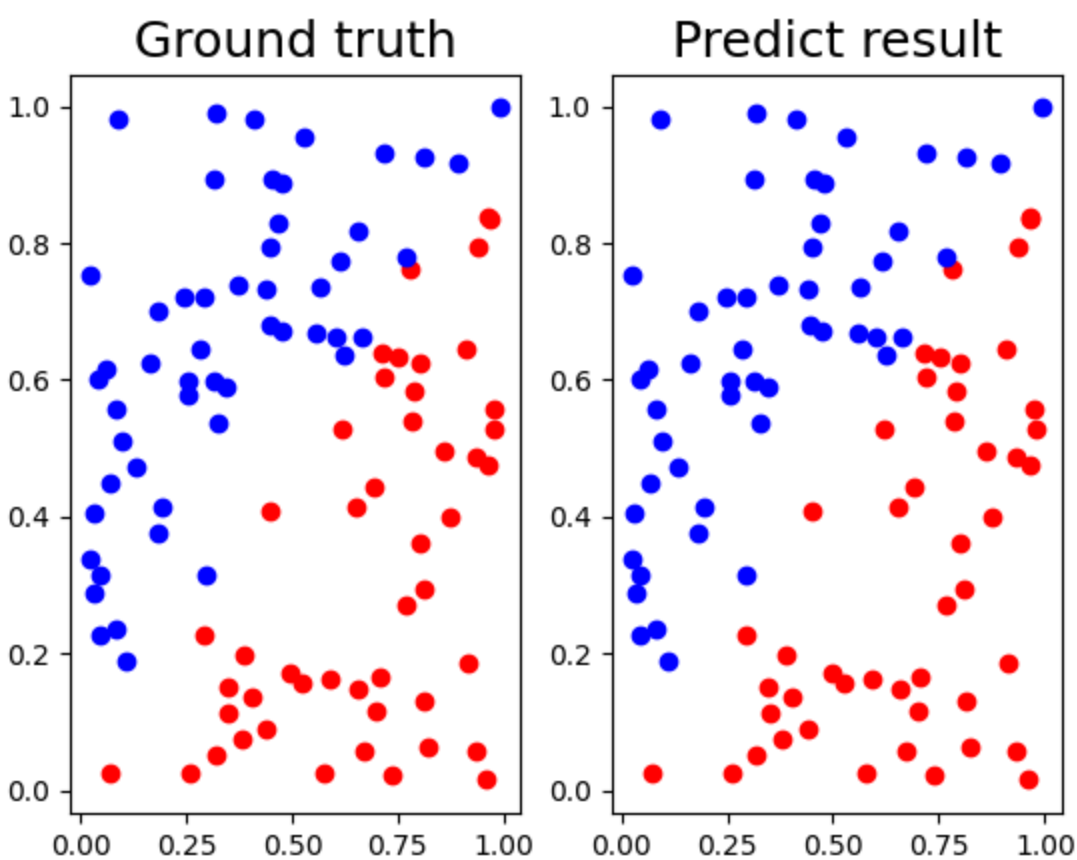
```
def backpropagation(self,y_gt):
        if self.activation=="sigmoid":
            act=self.sigmoid_derivative
        elif self.activation=="none":
            act=self.none_derivative
        elif self.activation=="relu":
            act=self.relu_derivative

        weight3_no_bias=self.weight3[:-1, :].copy()
        weight2_no_bias=self.weight2[:-1, :].copy()

        y_error=self.y-y_gt

        batch=y_gt.shape[0]
        self.y_delta=y_error*act(self.y)/batch

        hidden2_error=np.dot(self.y_delta,weight3_no_bias.T)
        self.hidden2_delta=hidden2_error*act(self.hidden2_output)

        hidden1_error=np.dot(self.hidden2_delta,weight2_no_bias.T)
        self.hidden1_delta=hidden1_error*act(self.hidden1_output)
        return np.mean((self.y-y_gt)**2)
    def update(self):
        if self.optimizer=="momentum":
            # v=beta*v-lr*L(theta)
            # w=w+v
            self.v_weight3=self.v_weight3*self.beta-
self.learning_rate*np.dot(self.hidden2_bias.T,self.y_delta)
            self.weight3+=self.v_weight3
            self.v_weight2=self.v_weight2*self.beta-
```

```
self.learning_rate*np.dot(self.hidden1_bias.T,self.hidden2_delta)
            self.weight2+=self.v_weight2
            self.v_weight1=self.v_weight1*self.beta-
self.learning_rate*np.dot(self.x_bias.T,self.hidden1_delta)
            self.weight1+=self.v_weight1
        else :
            self.weight3-
=self.learning_rate*np.dot(self.hidden2_bias.T,self.y_delta)
            self.weight2-
=self.learning_rate*np.dot(self.hidden1_bias.T,self.hidden2_delta)
            self.weight1-
=self.learning_rate*np.dot(self.x_bias.T,self.hidden1_delta)
```

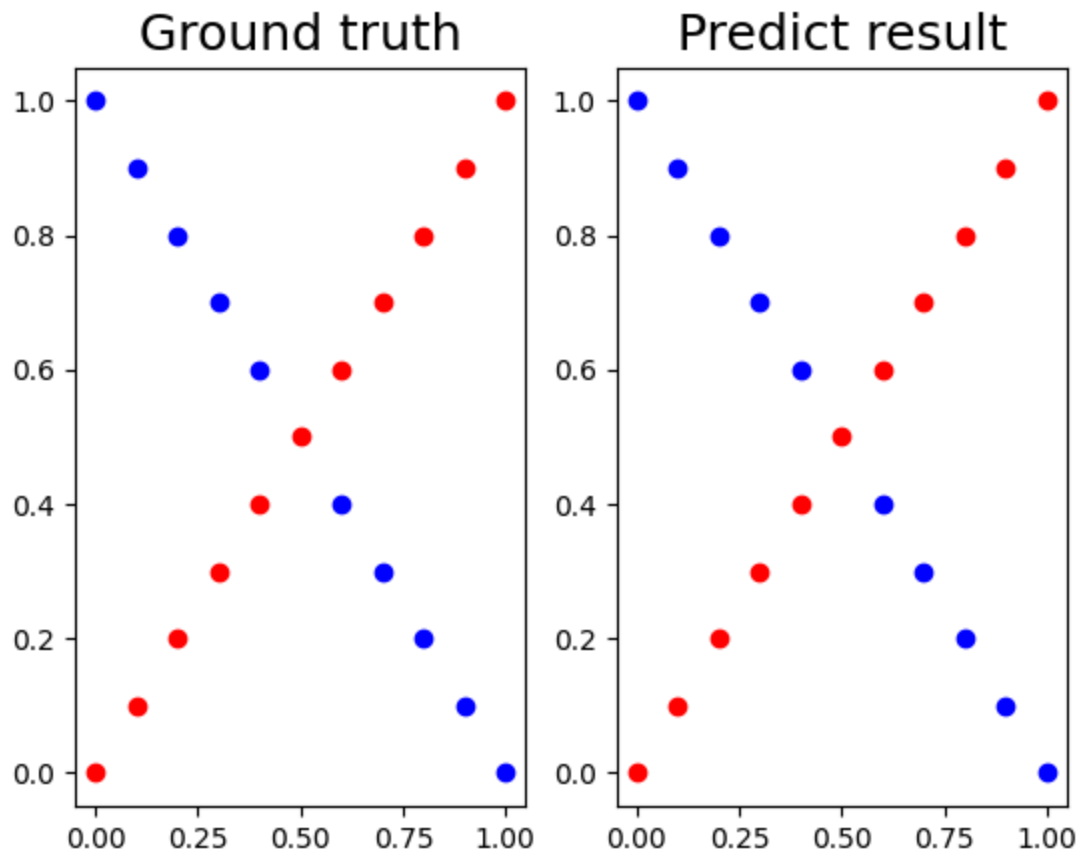
# 3.Result of your testing

## A.Screenshot and comparison figure

- Linear Data




- XOR Data




## B. Show the accuracy of your prediction

- Setup:
  - Learning rate=0.1
  - Hidden_unit:4
  - Optimizer=SGD
  - Activation function=sigmoid
- linear

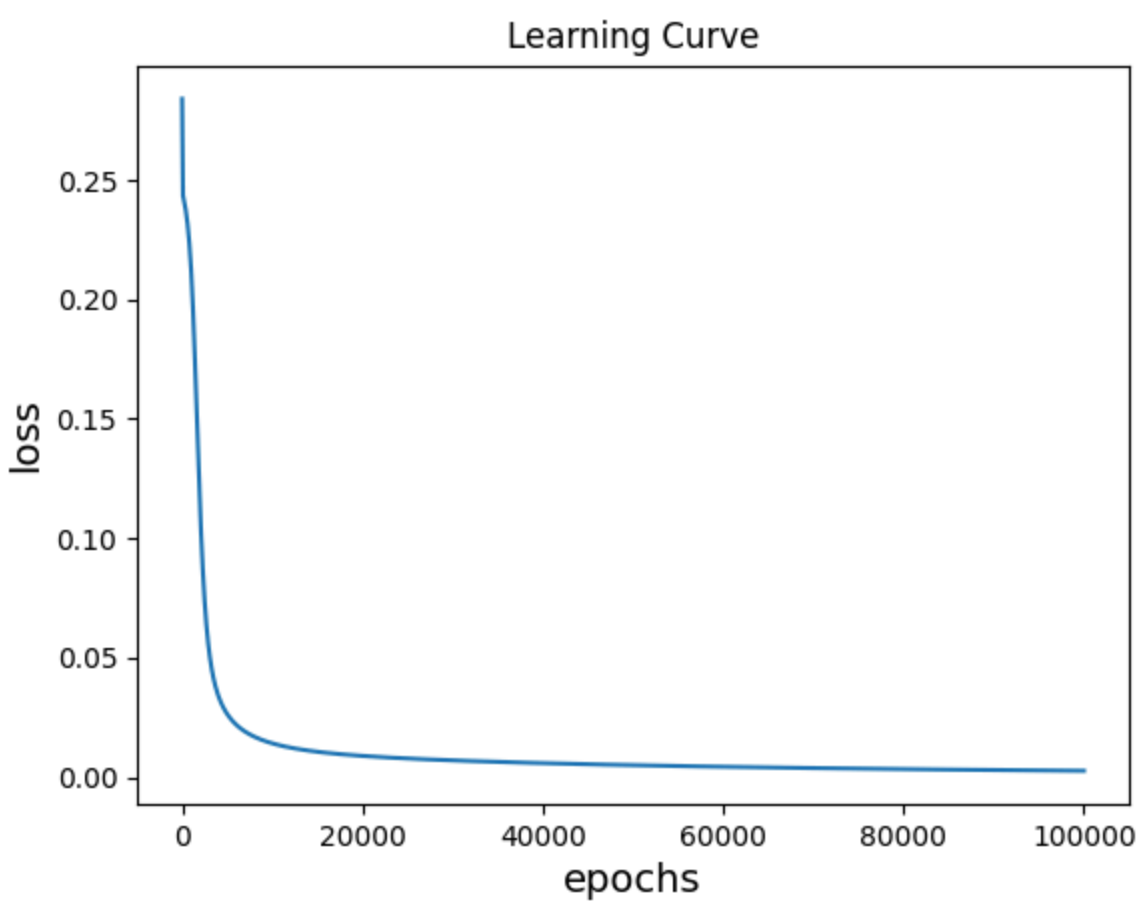
```
Iter90 |        Ground truth: 1.0 |        prediction: 0.99984 |
Iter91 |        Ground truth: 1.0 |        prediction: 0.99985 |
Iter92 |        Ground truth: 1.0 |        prediction: 0.88373 |
Iter93 |        Ground truth: 1.0 |        prediction: 0.99984 |
Iter94 |        Ground truth: 1.0 |        prediction: 0.99985 |
Iter95 |        Ground truth: 0.0 |        prediction: 0.00003 |
Iter96 |        Ground truth: 0.0 |        prediction: 0.00003 |
Iter97 |        Ground truth: 0.0 |        prediction: 0.00011 |
Iter98 |        Ground truth: 0.0 |        prediction: 0.06023 |
Iter99 |        Ground truth: 1.0 |        prediction: 0.99969 |
loss=0.00261 accuracy=100.00%
```

- XOR

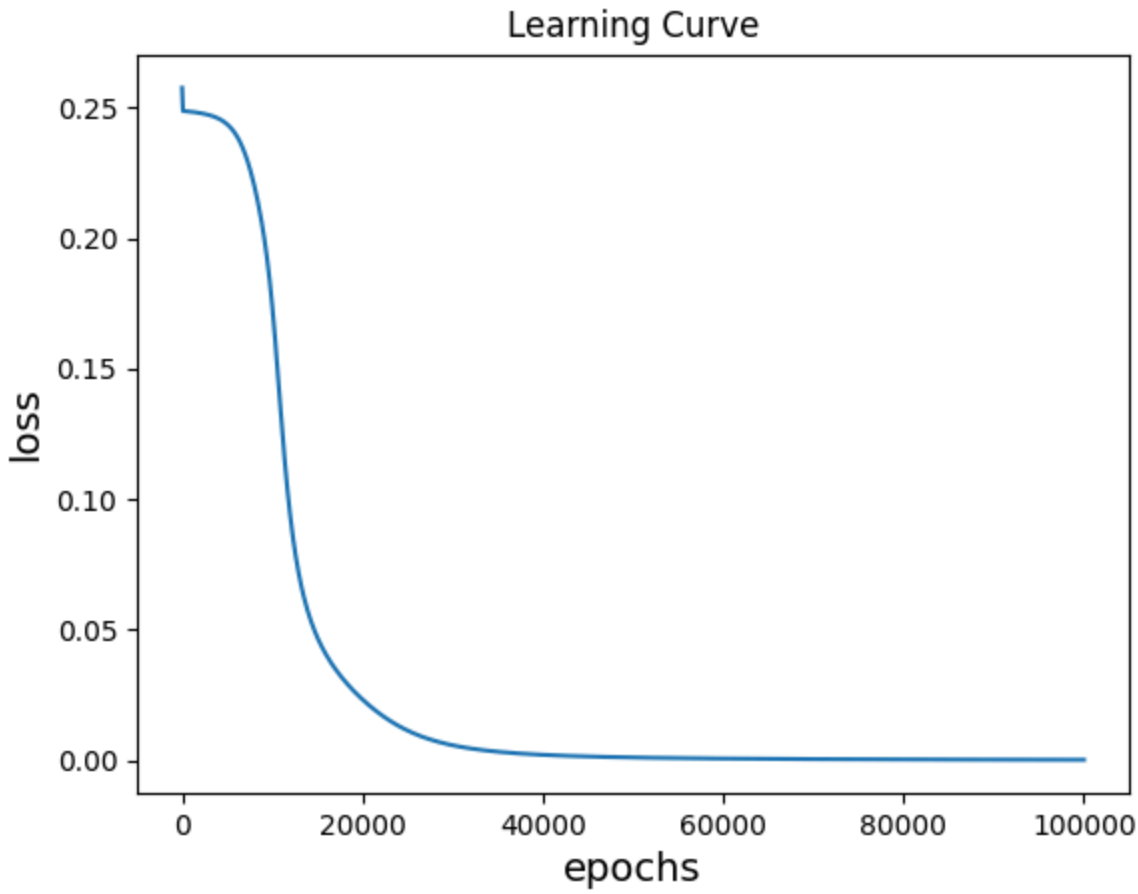
```
Iter11 |        Ground truth: 0.0 |        prediction: 0.01993 |
Iter12 |        Ground truth: 1.0 |        prediction: 0.96478 |
Iter13 |        Ground truth: 0.0 |        prediction: 0.00604 |
Iter14 |        Ground truth: 1.0 |        prediction: 0.99990 |
Iter15 |        Ground truth: 0.0 |        prediction: 0.00214 |
Iter16 |        Ground truth: 1.0 |        prediction: 0.99996 |
Iter17 |        Ground truth: 0.0 |        prediction: 0.00099 |
Iter18 |        Ground truth: 1.0 |        prediction: 0.99996 |
Iter19 |        Ground truth: 0.0 |        prediction: 0.00057 |
Iter20 |        Ground truth: 1.0 |        prediction: 0.99995 |
loss=0.00026 accuracy=100.00%
```

## C.Learning Curve

- Setup:
  - Learning rate=0.1
  - Hidden_unit:4
  - Optimizer=SGD
  - Activation function=sigmoid
- linear



- XOR




## Anything you want to present

- Training log
  - linear

```
epoch: 50000 loss: 0.00496144437418777
epoch: 55000 loss: 0.004608361415355065
epoch: 60000 loss: 0.004292571386292989
epoch: 65000 loss: 0.0040083437803173205
epoch: 70000 loss: 0.003751214431410402
epoch: 75000 loss: 0.003517588576488222
epoch: 80000 loss: 0.003304506839172902
epoch: 85000 loss: 0.0031094966767652456
epoch: 90000 loss: 0.0029304710580791383
epoch: 95000 loss: 0.0027656549527850975
epoch: 100000 loss: 0.0026135294161294459
```

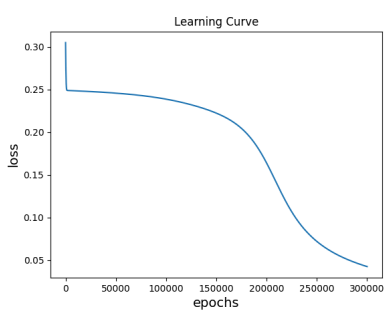
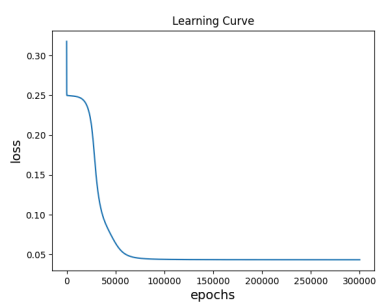
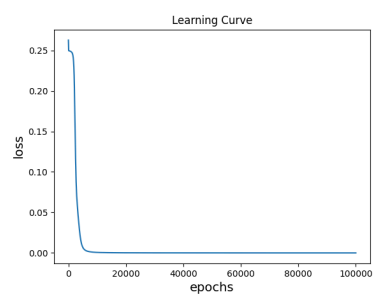
- XOR

```
epoch: 50000 loss: 0.0011414533323224594
epoch: 55000 loss: 0.0008958819143008505
epoch: 60000 loss: 0.0007276504838140017
epoch: 65000 loss: 0.0006068351449757904
epoch: 70000 loss: 0.0005167531842774445
epoch: 75000 loss: 0.0004475132329665332
epoch: 80000 loss: 0.0003929442312269136
epoch: 85000 loss: 0.00034902885670625133
epoch: 90000 loss: 0.00031305619377709873
epoch: 95000 loss: 0.00028313993319545826
epoch: 100000 loss: 0.00025793238730568464
```

# 4.Discussion

## A.Try different learning rates

- Setup
  - data: XOR
  - Hidden_unit:4
  - Optimizer=SGD
  - Activation function=sigmoid

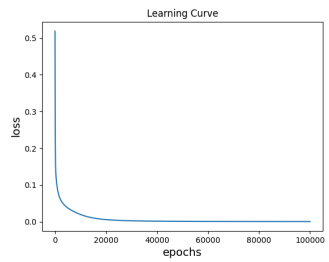
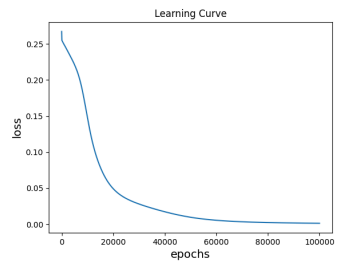
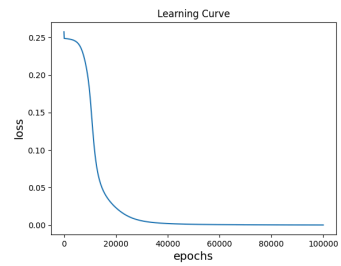
| lr=0.01 | lr=0.1 | lr=1 |
|---|---|---|
|   |   |   |

- 從三張 learning curve 可以觀察到，在 activation function 為 sigmoid 的情況下，不同的 learning rate 會明顯影響模型的收斂速度與訓練效率。較大的 learning rate 能帶來更快的收斂。

## B.Try different numbers of hidden units

- Setup
  - XOR data
  - learning rate=0.1

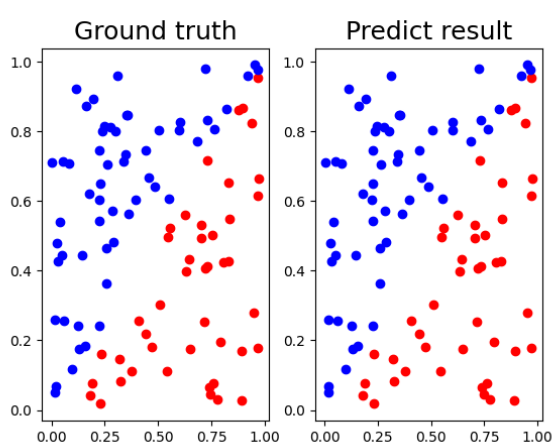
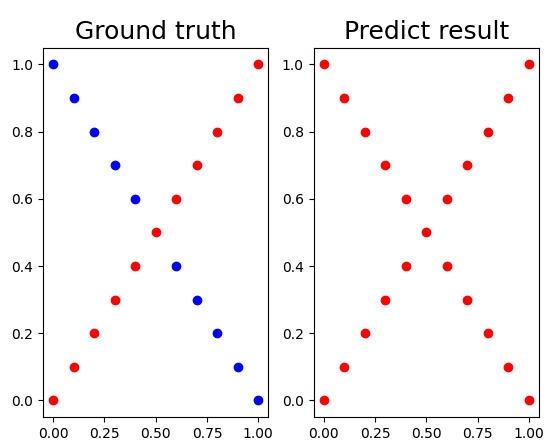
- Optimizer=SGD
- Activation function=sigmoid

| hidden_units=100 | 10 | 4 |
|---|---|---|
|   |   |   |

- 從這三張 learning curve 可以看出，在 hidden_units = 100、10、4 的情況下，模型的 loss 都有明顯下降，並且最後都逐漸收斂到接近 0。這代表不論hidden layer神經元數量多或少，模型都具有學習能力。

## Try without activation function

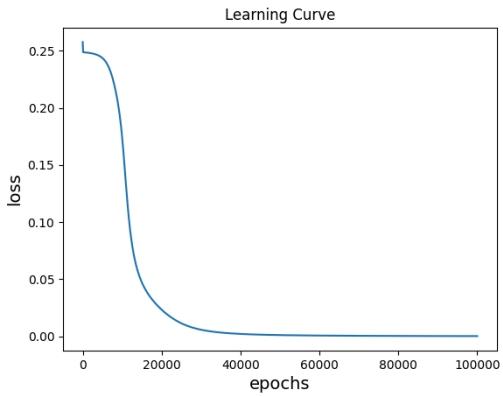
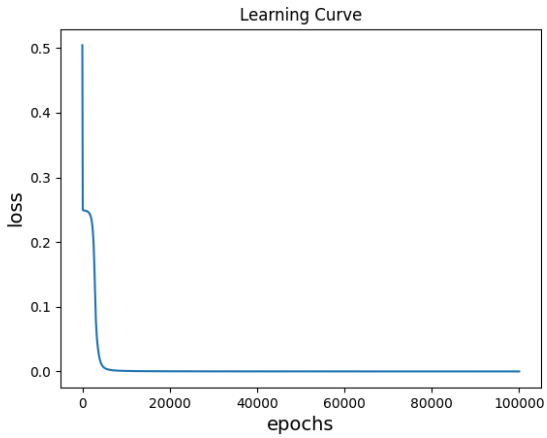
- Setup
  - hidden units=4
  - learning rate=0.1
  - Optimizer=SGD
- w/o activation

| linear | XOR |
|---|---|
|   |   |

- 在沒有 activation function 的情況下，模型在 linear 資料上是可以正確分類，但在 XOR 資料上無法正確分類。原因是拿掉 activation function 後，整個神經網路不論堆疊幾層，本質上仍然只是線性模型，而若只能處理線性資料，則XOR這種非線性資料會無法正確學習。

## Anythig you want to share

- with optimizer(Momentum) vs without optimizer
- Setup
  - XOR data
  - learning rate=0.1
  - hidden_unit=4
  - Activation function=sigmoid

| with SGD | with momentum | |
|---|---|---|
|   |   | |

- 使用 momentum 時，loss 下降較平穩，曲線也較平滑，這說明 momentum 的作用主要在於讓參數更新更穩定、減少震盪。

# 5.Question

- Q.What is the purpose of activation functions?
- A.若不使用Activation function，每一層只會做線性運算，則在處理非線性資料，例如XOR dataset時，會無法正確預測，模型的能力就會大幅降低。
- Q.What might happen if the learning rate is too large or too small?
- A.learning rate如果太大，會導致更新幅度會過大，會跳過最佳解，使得讓loss上下起伏過大，甚至無法收斂，而learning rate如果設置太小，會導致更新幅度很小，導致會需要更多epochs才能收斂。
- Q.What is the purpose of weights and biases in a neural network?
- A.weight 可以決定每個特徵的重要程度，模型會透過調整weight，來讓哪些特徵是比較重要、哪些是不重要的，而bias提供一個額外的常數，讓模型增加一點不確定性。

# 6.Extra

## Implement different optimizers.

- Momentum參數

```
    self.beta=0.9
    self.v_weight1=np.zeros_like(self.weight1)
    self.v_weight2=np.zeros_like(self.weight2)
    self.v_weight3=np.zeros_like(self.weight3)
```

- 實作

```
    self.v_weight3=self.v_weight3*self.beta-
self.learning_rate*np.dot(self.hidden2_bias.T,self.y_delta)
    self.weight3+=self.v_weight3
    self.v_weight2=self.v_weight2*self.beta-
self.learning_rate*np.dot(self.hidden1_bias.T,self.hidden2_delta)
    self.weight2+=self.v_weight2
    self.v_weight1=self.v_weight1*self.beta-
self.learning_rate*np.dot(self.x_bias.T,self.hidden1_delta)
    self.weight1+=self.v_weight1
```

## Implement different activation functions

- ReLU

```
    def relu(self,x):
        return np.maximum(0,x)
    def relu_derivative(self,x):
        return (x>0).astype(float)
```